

JavaScript: основи

Головна ідея: JS-код складається з інструкцій (statements), які містять вирази (expressions) і літерали. Дані зберігаються у змінних (`let` / `const`) з примітивними типами (Number, String, Boolean, null, undefined). Розгалуження (`if` , тернарний оператор, `switch`) і цикли (`while` , `for`) керують потоком виконання; функції дозволяють повторно використовувати логіку.

1. Базова термінологія

Ключові поняття

Термін	Значення
Інструкція (statement)	завершена команда для машини, закінчується <code>;</code> . Наприклад: <code>a = b * 2;</code>
Вираз (expression)	посилання на змінну/значення або їх комбінацію з операторами (частина інструкції)
Літерал (literal)	значення, записане прямо в коді: <code>10</code> (числовий), <code>"текст"</code> (рядковий)
Вихідний код (source code)	текст програми, який інтерпретатор/компілятор перетворює на інструкції для машини
Алгоритм	послідовність кроків для досягнення мети

Підключення скрипта до HTML

```
<!-- Вбудований -->
<script>
  console.log("Hello, world");
</script>

<!-- Зовнішній (рекомендовано) -->
<script src="my-script.js" defer></script>
```

defer — скрипт виконається лише після повного завантаження HTML, не блокуючи відображення сторінки.

Суворий режим і виведення

```
'use strict'; // на початку файлу – вмикає сучасні правила, ловить приховані помилки

console.log("JavaScript is awesome!"); // виведення в консоль DevTools
```

2. Змінні та типи даних

Оголошення

```
const age = 20;           // не можна перевизначити
let username = "Mango";  // можна перевизначити

username = "Poly"; // OK для let
// age = 25;       // ✗ TypeError для const
```

	const	let
Перевизначення значення	неможливе	можливе
Коли використовувати	за замовчуванням, якщо значення не змінюється	лічильники, тимчасові значення

Іменування (camelCase)

- літери, цифри, `_` , `$` ; перший символ — не цифра
- чутливість до регістру: `user` $\neq$ `User`
- перше слово — з малої, кожне наступне — з великої: `userAge` , `getUserData`

Примітивні типи даних

Тип	Приклад	Особливість
Number	<code>20</code> , <code>3710.84</code>	ціле й дробове — один тип; розділювач — крапка
String	<code>'Mango'</code> , <code>"text"</code>	послідовність символів у лапках
Boolean	<code>true</code> , <code>false</code>	без лапок; <code>"true"</code> в лапках — це рядок, не буль
null	<code>let x = null;</code>	явно вказана відсутність значення
undefined	<code>let x;</code>	значення ще не задано (автоматично)

typeof

```
typeof 17;           // "number"
typeof "text";       // "string"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof null;         // "object"  ⚠ історична помилка мови, null – не об'єкт
```

Типові помилки: звернення до неоголошеної змінної (`ReferenceError`), звернення до змінної **до** її оголошення через `let / const` , спроба перевизначити `const` .

3. Рядки

Конкатенація та шаблонні рядки

```
// Конкатенація (+)
const message = "Poly is " + age + " years old!";

// Шаблонний рядок (зворотні лапки ``) – читабельніше
const greeting = `Welcome ${guestName}, room ${roomNumber}!`;
```

Порядок операндів має значення: `1 + "2" → "12"` , але `1 + 2 + "4" → "34"` (спершу `1+2=3`, потім конкатенація з `"4"`).

Довжина, індексація, незмінність

```
const product = "Repair droid";
product.length;           // 12
product[0];                // 'R'
product[product.length - 1]; // 'd' (останній символ)

product[2] = "0"; // ✖ не працює – рядки незмінні (immutable)
product = "Repair dr0id"; // ✔ треба створити новий рядок
```

4. Оператори порівняння та рівності

Оператор	Значення
<code>></code> <code><</code> <code>>=</code> <code><=</code>	більше / менше / більше-дорівнює / менше-дорівнює
<code>==</code> <code>/</code> <code>!=</code>	несувора (не)рівність — з приведенням типів (уникати)
<code>===</code> <code>/</code> <code>!==</code>	сувора (не)рівність — без приведення типів (рекомендовано)

```
5 == "5"; // true  (рядок приводиться до числа)
5 === "5"; // false (різні типи → не рівні)
1 == true; // true
1 === true; // false
```

5. Перетворення типів

Функція	Приклад	Результат
<code>String(value)</code>	<code>String(5)</code> , <code>String(null)</code>	<code>"5"</code> , <code>"null"</code>
<code>Number(value)</code>	<code>Number(true)</code> , <code>Number(null)</code> , <code>Number("Jacob")</code>	<code>1</code> , <code>0</code> , <code>NaN</code>
<code>Boolean(value)</code>	див. розділ 8 (логічне перетворення)	—
<code>Number.parseInt(str)</code>	<code>parseInt("12qwe74")</code>	<code>12</code> (ціле)
<code>Number.parseFloat(str)</code>	<code>parseFloat("12.46qwe")</code>	<code>12.46</code> (дробове)
<code>num.toFixed(n)</code>	<code>(0.1+0.2).toFixed(1)</code>	<code>"0.3"</code> (рядок!)

Класичний нюанс: `0.1 + 0.2 === 0.3 → false` через двійкове представлення дробів у пам'яті. Рішення: `toFixed()` або множення/ділення на 10/100 перед операцією.

6. Функції

Оголошення, виклик, параметри

```
function multiply(x, y, z) { // x, y, z – параметри
  return x * y * z;         // return – повертає значення й завершує функцію
}

multiply(2, 3, 5); // 2, 3, 5 – аргументи; результат: 30
```

Термін	Значення
Параметри	локальні змінні в оголошенні функції — placeholder-и
Аргументи	реальні значення, що передаються під час виклику
return	повертає значення і одразу припиняє виконання функції (код після return не виконується)
Без return	функція повертає <code>undefined</code>

Область видимості (scope)

- Глобальна** — змінна поза будь-яким блоком, доступна всюди
- Локальна/блокова** — змінна всередині `{ }` (функція, if, цикл), доступна лише в цьому блоці й вкладених

- Вкладений блок бачить змінні "батьківських" блоків, але не навпаки

7. Розгалуження

Конструкція	Коли використовувати
<code>if</code>	одна умова, універсальний варіант
<code>if...else</code>	дві альтернативи
<code>else if</code>	кілька послідовних умов (перевіряються згори вниз, перша істинна — виконується, решта ігноруються)
Тернарний оператор <code>? :</code>	коротке присвоєння/повернення значення, рівно 2 сценарії
<code>switch</code>	порівняння одного виразу з багатьма варіантами (тільки сувора рівність <code>===</code>)

```
// Тернарний оператор
const type = age >= 18 ? 'adult' : 'child';

// switch
switch (fruit) {
  case 'apple':
    console.log('Apple selected');
    break;
  default:
    console.log('Unknown fruit');
}
```

switch: не забувай `break` після кожного `case` (інакше — "провалювання" в наступний блок); `default` — один, завжди останній, без `break`.

8. Логічні оператори

Логічне перетворення (Boolean)

6 значень, що приводяться до false: `0` , `""` , `NaN` , `null` , `undefined` , `false` . Усе інше → `true` .

&& (І), || (АБО), ! (НІ)

Оператор	Логіка	Приклад
<code>&&</code>	true, якщо ОБИДВА операнди true; повертає перший хибний АБО останній (правий) операнд	<code>5 && "hello" → "hello"</code> ; <code>0 && "hello" → 0</code>
<code> </code>	true, якщо ХОЧА Б ОДИН true; повертає перший істинний операнд	<code>false 5 → 5</code> ; <code>0 false → false</code>
<code>!</code>	інвертує значення (унарний, один операнд)	<code>!true → false</code> ; <code>!0 → true</code>

```
const isAdult = age >= 18 && age < 65; // обидві умови мають бути true
const hasAccess = type === "pro" || type === "vip"; // достатньо однієї
const canChat = isOnline && !isBlocked; // онлайн І не заблокований
```

9. Методи рядків

Метод	Що робить	Приклад
<code>.length</code>	довжина рядка (властивість, без дужок)	<code>"hello".length → 5</code>
<code>.slice(start, end)</code>	підрядок від start до end (не включно)	<code>"Jacob".slice(0,3) → "Jac"</code>
<code>.toLowerCase()</code> / <code>.toUpperCase()</code>	зміна регістру (новий рядок)	<code>"Mango".toLowerCase() → "mango"</code>
<code>.includes(sub)</code>	чи містить підрядок (чутливо до регістру)	<code>"Jacob".includes("cob") → true</code>
<code>.startsWith(sub)</code> / <code>.endsWith(sub)</code>	перевірка початку/кінця рядка	<code>"app.js".endsWith(".js") → true</code>
<code>.indexOf(sub)</code>	індекс першого входження або <code>-1</code>	<code>"hello".indexOf("l") → 2</code>
<code>.trim()</code>	прибирає пробіли з початку/кінця	<code>" hi ".trim() → "hi"</code>

Жоден із цих методів **не змінює** вихідний рядок — усі повертають новий (рядки незмінні, див. розділ 3).

10. Цикли

Цикл	Особливість
<code>while</code>	перевіряє умову перед кожною ітерацією; тіло може не виконатись жодного разу
<code>do...while</code>	виконує тіло хоча б один раз , потім перевіряє умову
<code>for</code>	має лічильник: ініціалізація; умова; пост-вираз — усе в одному рядку

```
// while
let count = 0;
while (count < 10) {
  console.log(count);
  count += 1;
}

// do...while – виконається хоч раз, навіть якщо умова хибна від початку
do {
  console.log(count);
  count += 1;
} while (count < 5);

// for
for (let i = 0; i <= 20; i += 5) {
  console.log(i); // 0, 5, 10, 15, 20
}
```

Інкремент / декремент

Форма	Поведінка
Префіксний <code>++x</code>	спочатку збільшує, потім використовує нове значення
Постфіксний <code>x++</code>	спочатку використовує поточне значення, потім збільшує

break

```
for (let i = 0; i < 10; i++) {
  if (i === 5) break; // перериває ТІЛЬКИ цикл, не функцію
}
// return перериває і цикл, і функцію одразу
```

11. Ключові методи/функції — швидкий довідник

Метод / оператор	Категорія	Призначення
<code>console.log()</code>	виведення	вивід значення в консоль DevTools
<code>typeof</code>	типи	визначає тип даних значення
<code>String()</code> / <code>Number()</code> / <code>Boolean()</code>	перетворення типів	явне приведення до типу
<code>Number.parseInt()</code> / <code>parseFloat()</code>	перетворення типів	рядок → число (ціле / дробове)
<code>.toFixed(n)</code>	числа	округлення до n знаків після крапки (повертає рядок)
<code>Math.floor/ceil/round/max/min/random</code>	числа	математичні операції
<code>.length</code>	рядки	довжина рядка
<code>.slice()</code>	рядки	витягнути підрядок
<code>.includes()</code> / <code>.indexOf()</code>	рядки	пошук підрядка
<code>.trim()</code>	рядки	прибрати зайві пробіли
<code>.toLowerCase()</code> / <code>.toUpperCase()</code>	рядки	зміна регістру
<code>&&</code> / <code> </code> / <code>!</code>	логіка	І / АБО / заперечення
<code>===</code> / <code>!==</code>	порівняння	сувора (не)рівність (рекомендовано)
<code>if</code> / <code>else if</code> / <code>else</code>	розгалуження	умовне виконання коду
<code>switch</code> / <code>case</code> / <code>break</code> / <code>default</code>	розгалуження	порівняння одного виразу з багатьма варіантами
<code>? :</code>	розгалуження	тернарний оператор (коротке присвоєння)
<code>while</code> / <code>do...while</code> / <code>for</code>	цикли	багаторазове виконання коду
<code>break</code> / <code>return</code>	цикли/функції	перервати цикл / перервати цикл і функцію одразу